



PYTHON

PCAD – CERTIFIED ASSOCIATE DATA ANALYST WITH PYTHON
CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

2.1 Core Python Proficiency

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Aplicar sintaxis de Python y estructuras de control para resolver problemas relacionados con datos.**
- A. Uso correcto de la sintaxis básica de Python: variables, ámbitos y tipos de datos
 - Variables: permiten almacenar información para su procesamiento y análisis.
 - Ejemplo: edad = 25
 - Tipos de datos principales:
 - int → números enteros
 - float → números decimal
 - esstr → texto
 - bool → valores lógicos (True / False)
 - list, dict, tuple → colecciones de datos

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Aplicar sintaxis de Python y estructuras de control para resolver problemas relacionados con datos.**
- A. Uso correcto de la sintaxis básica de Python:
 - **Ámbito de las variables (scope):** define dónde puede utilizarse una variable.
 - Local: creada dentro de una función
 - Global: accesible desde cualquier parte del programa
 - **Buenas prácticas:**
 - Usar nombres descriptivos
 - Mantener consistencia en el código
 - Evitar redefinir variables innecesariamente
 - **Aplicación en datos:** facilita almacenar, transformar y organizar conjuntos de datos para su análisis.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Aplicar sintaxis de Python y estructuras de control para resolver problemas relacionados con datos.**
- B. Implementación de estructuras de control: bucles y condicionales.
 - Condicionales (if, elif, else): permiten tomar decisiones según condiciones específicas.
 - Bucles (for, while): automatizan tareas repetitivas sobre datos.
 - Uso combinado: los programas pueden filtrar, validar y procesar datos dinámicamente.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- **Aplicar sintaxis de Python y estructuras de control para resolver problemas relacionados con datos.**
- B. Implementación de estructuras de control: bucles y condicionales.
 - Control del flujo:
 - break → detiene un bucle
 - continue → salta a la siguiente iteración
 - Aplicación en análisis de datos:
 - Recorrer registros
 - Detectar valores específicos
 - Automatizar cálculos y transformaciones
 - Validar información antes de procesarla

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Analizar y crear funciones en Python.**
- A. Diseñar funciones con propósito claro usando argumentos posicionales y por nombre
 - Funciones: bloques reutilizables de código que realizan tareas específicas.
 - Mejoran la organización, reutilización y mantenimiento del programa.
 - Argumentos posicionales (indexed): se asignan según el orden.
 - Argumentos por nombre (keyword): se asignan indicando el parámetro.
 - Las funciones deben tener:
 - Nombre descriptivo
 - Objetivo específico
 - Parámetros claros
 - Resultado definido (return)
 - Aplicación en datos: crear procesos reutilizables para limpieza, transformación y análisis de información.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Analizar y crear funciones en Python.**
- B. Diferenciar argumentos opcionales y obligatorios
 - Argumentos obligatorios: necesarios para ejecutar la función.
 - Argumentos opcionales: poseen valores por defecto.
 - Los argumentos opcionales aumentan la flexibilidad y reutilización del código.
 - Permiten adaptar funciones a distintos escenarios sin duplicar lógica.
 - Aplicación práctica: funciones configurables para análisis de datos, filtros y reportes personalizados.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Evaluar y navegar el ecosistema de Python para Ciencia de Datos**
- A. Identificar bibliotecas y herramientas esenciales para Ciencia de Datos
 - NumPy: operaciones matemáticas y arreglos numéricos.
 - Pandas: manipulación y análisis de datos tabulares.
 - Matplotlib / Seaborn: visualización de datos.
 - Scikit-learn: machine learning y modelos predictivos.
 - Jupyter Notebook / Google Colab: entornos interactivo para análisis y documentación.
 - SciPy: cálculos científicos y estadísticos.
 - Estas herramientas permiten:
 - Procesar grandes volúmenes de datos
 - Automatizar análisis
 - Crear modelos y visualizaciones

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Evaluar y navegar el ecosistema de Python para Ciencia de Datos**
- B. Evaluar la utilidad de recursos según el escenario de análisis
 - La elección de herramientas depende de:
 - Tipo de datos
 - Complejidad del análisis
 - Rendimiento requerido
 - Facilidad de uso
 - Ejemplos:
 - Pandas → análisis tabular
 - NumPy → cálculos numéricos intensivos
 - Scikit-learn → predicción y clasificación
 - Matplotlib → gráficos personalizados
 - Evaluar correctamente las herramientas mejora:
 - Eficiencia
 - Escalabilidad
 - Calidad de los resultados

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Organizar y manipular datos con estructuras básicas de Python**
- A. Uso efectivo de tuplas, conjuntos, listas, diccionarios y cadenas
 - Listas (list): colecciones ordenadas y modificables.
 - Tuplas (tuple): colecciones ordenadas e inmutables.
 - Conjuntos (set): elementos únicos sin orden.
 - Diccionarios (dict): pares clave–valor.
 - Strings (str): manejo y procesamiento de texto.
 - Estas estructuras permiten:
 - Organizar información
 - Buscar datos rápidamente
 - Transformar contenidos
 - Gestionar colecciones complejas

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Organizar y manipular datos con estructuras básicas de Python**
- B. Resolver tareas complejas eligiendo la estructura adecuada
 - La selección correcta optimiza:
 - Rendimiento
 - Legibilidad
 - Eficiencia del programa
 - Ejemplos de uso:
 - Listas → secuencias de datos
 - Diccionarios → registros estructurados
 - Sets → eliminación de duplicados
 - Tuplas → datos constantes
 - Aplicación en datos:
 - Limpieza de información
 - Agrupación de registros
 - Validación y filtrado
 - Procesamiento masivo

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Explicar e implementar buenas prácticas de scripting en Python**
- A. Comprender y aplicar las guías PEP 8
 - PEP 8: estándar oficial de estilo para código Python.
 - Busca mejorar:
 - Legibilidad
 - Consistencia
 - Mantenimiento del código
 - Principales recomendaciones:
 - Indentación de 4 espacios
 - Nombres descriptivos
 - Líneas cortas y claras
 - Espacios adecuados entre operadores y funciones
 - Beneficios:
 - Trabajo colaborativo más eficiente
 - Código más limpio y profesional

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Explicar e implementar buenas prácticas de scripting en Python**
- B. Comprender y utilizar PEP 257 para documentación
 - PEP 257: estándar para escribir docstrings en Python.
 - Los docstrings describen:
 - Objetivo de funciones y clases
 - Parámetros
 - Valores de retorno
 - Beneficios:
 - Facilita mantenimiento y reutilización
 - Mejora comprensión del código
 - Ayuda a generar documentación automática
 - Aplicación profesional: proyectos colaborativos y desarrollo de software orientado a datos.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

2.2 Module Management and Exception Handling

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Importar módulos y gestionar paquetes de Python usando PIP
- A. Aplicar diferentes tipos de importación de módulos
 - Importación estándar: importa el módulo completo.
 - Importación selectiva: importa elementos específicos.
 - Importación con alias: simplifica nombres largos.
 - Cada método permite organizar mejor el código según la necesidad del proyecto.
 - Aplicación práctica: facilitar acceso a funciones matemáticas, análisis de datos y visualización.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- **Importar módulos y gestionar paquetes de Python usando PIP**
- B. Comprender la importación desde diferentes fuentes
 - Biblioteca estándar de Python: módulos integrados como os, math y datetime.
 - Paquetes instalados con PIP: bibliotecas externas como:
 - Pandas
 - Numpy
 - Matplotlib
 - Módulos locales: archivos creados por el desarrollador para reutilizar funciones propias.
 - Comprender el origen de los módulos permite:
 - Mejor organización
 - Reutilización del código
 - Escalabilidad de proyectos

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Importar módulos y gestionar paquetes de Python usando PIP
- C. Identificar e importar módulos adecuados para tareas específicas
 - La selección del módulo depende del problema a resolver.
 - Ejemplos:
 - math → operaciones matemáticas
 - pandas → análisis de datos
 - random → generación aleatoria
 - matplotlib → gráficos
 - os → manejo de archivos y sistema
 - Elegir correctamente los módulos mejora:
 - Eficiencia
 - Productividad
 - Calidad del desarrollo

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Importar módulos y gestionar paquetes de Python usando PIP
- D. Gestionar paquetes de Python usando PIP
 - PIP: administrador de paquetes oficial de Python.
 - Comandos principales:
 - Instalar paquetes → `pip install`
 - Actualizar paquetes → `pip install --upgrade`
 - Eliminar paquetes → `pip uninstall`
 - Permite administrar dependencias y mantener entornos de desarrollo actualizados.
 - Aplicación profesional: instalación de herramientas de análisis, machine learning y visualización de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Aplicar manejo básico de excepciones y mantener la robustez de scripts
- A. Implementar técnicas básicas de manejo de excepciones
 - Las excepciones permiten controlar errores durante la ejecución del programa.
 - Python utiliza bloques:
 - try
 - except
 - Finally
 - Beneficios:
 - Evitar que el programa se detenga inesperadamente
 - Mejorar estabilidad y confiabilidad
 - Controlar situaciones imprevistas
 - Aplicación práctica: validación de datos, lectura de archivos y control de entradas del usuario.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Aplicar manejo básico de excepciones y mantener la robustez de scripts
- B. Predecir errores comunes y desarrollar estrategias de manejo
 - Errores frecuentes en Python:
 - `SyntaxError` → errores de sintaxis
 - `TypeError` → tipos incompatibles
 - `ValueError` → valores inválidos
 - `IndexError` → acceso fuera de rango
 - `FileNotFoundError` → archivo inexistente

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Aplicar manejo básico de excepciones y mantener la robustez de scripts
- B. Predecir errores comunes y desarrollar estrategias de manejo
 - Estrategias recomendadas:
 - Validar datos antes de procesarlos
 - Usar bloques try-except
 - Implementar mensajes claros de error
 - Probar diferentes escenarios
 - Anticipar errores mejora:
 - Experiencia del usuario
 - Calidad del software
 - Mantenimiento del código

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Aplicar manejo básico de excepciones y mantener la robustez de scripts
- C. Interpretar mensajes de error para diagnosticar y resolver problemas
 - Los mensajes de error ayudan a identificar:
 - Tipo de excepción
 - Línea del problema
 - Causa del fallo
 - Elementos importantes del traceback:
 - Archivo afectado
 - Número de línea
 - Descripción del error

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Aplicar manejo básico de excepciones y mantener la robustez de scripts
- C. Interpretar mensajes de error para diagnosticar y resolver problemas
 - Buenas prácticas:
 - Leer cuidadosamente el traceback
 - Corregir errores paso a paso
 - Registrar excepciones importantes
 - Resultado: scripts más robustos, confiables y fáciles de mantener en proyectos de análisis y procesamiento de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

2.3 Object-Oriented Programming for Data Modeling

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Aplicar programación orientada a objetos básica para estructurar y modelar datos
- A. Definir e instanciar clases para representar registros de datos estructurados
 - Las clases permiten modelar objetos y representar entidades del mundo real.
 - Una clase define:
 - Atributos (datos)
 - Métodos (comportamientos)
 - El método constructor `__init__()` inicializa los objetos al crearlos.
 - Una instancia es un objeto creado a partir de una clase.
 - Aplicación en datos: representar clientes, productos, registros o transacciones de forma organizada.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Aplicar programación orientada a objetos básica para estructurar y modelar datos
- B. Organizar atributos y comportamientos mediante constructores y métodos
 - Los atributos almacenan información del objeto.
 - Los métodos definen acciones o comportamientos.
 - Beneficios:
 - Código modular y reutilizable
 - Mejor organización lógica
 - Facilita mantenimiento y escalabilidad
 - Aplicación práctica: modelado de datos complejos y automatización de procesos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Aplicar programación orientada a objetos básica para estructurar y modelar datos
- C. Aplicar principios de encapsulamiento para gestionar el estado interno
 - El encapsulamiento protege los datos internos del objeto y controla su acceso.
 - Convenciones de nombres:
 - `_atributo` → protegido
 - `__atributo` → privado
 - Uso de getters y setters para acceder o modificar atributos de forma controlada.
 - Beneficios:
 - Mayor seguridad y control
 - Prevención de modificaciones incorrectas
 - Diseño más limpio y mantenible
 - Aplicación profesional: sistemas de gestión de datos y desarrollo de software estructurado.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Aplicar patrones orientados a objetos para mejorar la reutilización y claridad del código en flujos de análisis
 - A. Utilizar composición para agrupar modelos de datos relacionados
 - La composición permite construir objetos complejos utilizando otros objetos dentro de ellos.
 - Favorece la organización y reutilización de componentes relacionados.
 - Beneficios:
 - Mejor estructura de datos
 - Mayor modularidad
 - Relaciones claras entre entidades
 - Aplicación práctica: modelar usuarios, registros, respuestas y transacciones en sistemas de análisis de datos.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Aplicar patrones orientados a objetos para mejorar la reutilización y claridad del código en flujos de análisis
 - B. Extender clases base mediante herencia y sobrescritura de métodos
 - La herencia permite crear nuevas clases a partir de una clase base.
 - Las subclases reutilizan atributos y métodos existentes.
 - La sobrescritura de métodos permite personalizar comportamientos específicos.
 - Beneficios:
 - Reutilización de código
 - Reducción de duplicidad
 - Mayor flexibilidad en aplicaciones
 - Aplicación en análisis: diferentes exportadores para CSV, Excel o bases de datos.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Aplicar patrones orientados a objetos para mejorar la reutilización y claridad del código en flujos de análisis
 - C. Demostrar polimorfismo usando el mismo método en distintas subclases
 - El polimorfismo permite utilizar una misma interfaz para diferentes tipos de objetos.
 - Diferentes clases pueden responder al mismo método de manera distinta.
 - Beneficios:
 - Código más flexible y escalable
 - Integración sencilla de nuevos componentes
 - Simplificación de flujos de trabajo
 - Aplicación profesional: automatización de procesos de análisis y manejo de múltiples formatos de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Gestionar identidad y comparaciones de objetos en pipelines de datos
 - A. Utilizar variables de referencia y comprender comportamiento compartido e independiente
 - En Python, las variables de objetos almacenan referencias a posiciones en memoria.
 - Diferentes variables pueden apuntar al mismo objeto.
 - Para crear copias independientes se utilizan métodos como:
 - `copy()` → (listas, conjuntos y diccionarios)
 - `copy.deepcopy()` → (listas, tuplas, conjuntos y diccionarios)
 - Comprender este comportamiento es importante para:
 - Evitar modificaciones no deseadas
 - Mantener integridad de datos
 - Controlar mutaciones en listas y objetos
 - Aplicación práctica: manejo seguro de datos compartidos en análisis y procesamiento masivo.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Gestionar identidad y comparaciones de objetos en pipelines de datos
 - B. Comparar contenido de objetos usando `==`, `is` y `__eq__()`
 - `==` compara el contenido o valor de los objetos.
 - `is` compara si dos variables apuntan al mismo objeto en memoria.
 - El método especial `__eq__()` permite definir comparaciones personalizadas entre objetos.
 - Beneficios:
 - Comparaciones más precisas
 - Validación de registros
 - Control de duplicados y consistencia
 - Aplicación profesional: comparación de objetos en pipelines de datos, sistemas de validación y procesamiento automatizado.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

2.4 SQL for Data Analysts

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Realizar consultas SQL para recuperar y manipular datos
 - A. Componer y ejecutar consultas SQL para extraer datos de tablas
 - SQL (Structured Query Language) permite acceder y gestionar información en bases de datos.
 - La instrucción principal para consultar datos es SELECT.
 - Las consultas permiten:
 - Extraer información específica
 - Analizar registros
 - Generar reportes
 - Aplicación práctica: recuperación de datos empresariales, académicos o financieros.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Realizar consultas SQL para recuperar y manipular datos
 - B. Aplicar funciones y cláusulas SQL para manipular y filtrar datos
 - SQL incluye funciones para transformar y analizar información:
 - COUNT() → contar registros
 - AVG() → calcular promedios
 - SUM() → sumar valores
 - MAX() / MIN() → obtener máximos y mínimos
 - La cláusula WHERE filtra datos según condiciones.
 - Beneficios:
 - Filtrado preciso
 - Análisis más eficiente
 - Procesamiento de grandes volúmenes de datos

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Realizar consultas SQL para recuperar y manipular datos
 - C. Construir consultas usando SELECT, FROM, JOINS, WHERE, GROUP BY, HAVING, ORDER BY y LIMIT
 - SELECT → selecciona columnas.
 - FROM → indica la tabla origen.
 - JOINS → combinan datos de múltiples tablas:
 - INNER JOIN
 - LEFT JOIN
 - RIGHT JOIN
 - FULL JOIN
 - WHERE → filtra registros.
 - GROUP BY → agrupa datos.
 - HAVING → filtra grupos.
 - ORDER BY → ordena resultados.
 - LIMIT → restringe cantidad de filas.
 - Aplicación práctica: análisis de ventas, clientes, inventarios y métricas empresariales.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Realizar consultas SQL para recuperar y manipular datos
 - D. Analizar necesidades de recuperación de datos usando el conjunto **SFJWGHOL**
 - SFJWGHOL representa el orden lógico de cláusulas SQL:
 - S → SELECT
 - F → FROM
 - J → JOIN
 - W → WHERE
 - G → GROUP BY
 - H → HAVING
 - O → ORDER BY
 - L → LIMIT

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Realizar consultas SQL para recuperar y manipular datos
 - D. Analizar necesidades de recuperación de datos usando el conjunto **SFJWGHOL**
 - Comprender esta estructura permite:
 - Diseñar consultas eficientes
 - Resolver problemas de análisis
 - Optimizar recuperación de datos
 - Beneficios:
 - Consultas más claras y mantenibles
 - Mejor rendimiento en bases de datos
 - Resultados precisos según necesidades del negocio.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Ejecutar comandos fundamentales de SQL para crear, leer, actualizar y eliminar datos
 - A. Demostrar el uso de operaciones **CRUD** en SQL
 - CRUD representa las operaciones básicas de gestión de datos:
 - **Create** → crear registros
 - **Read** → leer o consultar datos
 - **Update** → actualizar información
 - **Delete** → eliminar registros
 - Estas operaciones son esenciales para administrar bases de datos relacionales.
 - Beneficios:
 - Gestión eficiente de información
 - Mantenimiento de bases de datos
 - Automatización de procesos empresariales
 - Aplicación práctica: sistemas de ventas, inventarios, clientes y plataformas de análisis de datos.

CHAPTER 2 – PROGRAMMING AND DATABASE SKILLS

- Ejecutar comandos fundamentales de SQL para crear, leer, actualizar y eliminar datos
 - B. Construir sentencias SQL para insertar, consultar, actualizar y eliminar datos
 - INSERT: agrega nuevos registros.
 - SELECT: recupera información.
 - UPDATE: modifica registros existentes.
 - DELETE: elimina registros.
 - Buenas prácticas:
 - Utilizar condiciones WHERE para evitar cambios masivos accidentales
 - Validar datos antes de modificar tablas
 - Mantener respaldos de información crítica
 - Resultado: manipulación segura y eficiente de datos en entornos de análisis y administración de bases de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Establecer conexiones a bases de datos usando Python
 - A. Comprender e implementar métodos para conectar bases de datos con Python
 - Python permite conectarse a bases de datos mediante bibliotecas especializadas.
 - Bibliotecas comunes:
 - sqlite3 → bases de datos SQLite
 - pymysql → bases de datos MySQL
 - psycopg2 → PostgreSQL
 - Las conexiones permiten:
 - Consultar datos
 - Insertar y actualizar registros
 - Automatizar procesos de análisis
 - Aplicación práctica: integración de Python con sistemas empresariales y plataformas de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Establecer conexiones a bases de datos usando Python
 - B. Analizar y resolver problemas comunes de conexión entre Python y bases de datos
 - Problemas frecuentes:
 - Credenciales incorrectas
 - Servidor inaccesible
 - Base de datos inexistente
 - Librerías no instaladas
 - Errores de permisos

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Establecer conexiones a bases de datos usando Python
 - B. Analizar y resolver problemas comunes de conexión entre Python y bases de datos
 - Estrategias de solución:
 - Validar usuario y contraseña
 - Verificar estado del servidor
 - Confirmar instalación de dependencias
 - Manejar excepciones con try-except
 - Buenas prácticas:
 - Cerrar conexiones después de usarlas
 - Evitar credenciales expuestas en código
 - Implementar manejo de errores y registros
 - Resultado: conexiones más seguras, confiables y eficientes para aplicaciones y análisis de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Ejecutar consultas SQL parametrizadas desde Python para interactuar de forma segura con bases de datos
 - A. Desarrollar y ejecutar consultas SQL parametrizadas en Python
 - Las consultas parametrizadas permiten enviar valores a SQL de manera segura sin concatenar texto directamente.
 - Se utilizan marcadores (? o %s) según la biblioteca empleada.
 - Beneficios:
 - Mayor seguridad
 - Separación entre datos y consultas
 - Mejor mantenimiento del código
 - Aplicación práctica: sistemas de autenticación, formularios y consultas dinámicas en aplicaciones de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Ejecutar consultas SQL parametrizadas desde Python para interactuar de forma segura con bases de datos
 - B. Evaluar ventajas de las consultas parametrizadas
 - Las consultas parametrizadas ayudan a prevenir SQL Injection, un ataque que inserta código malicioso en consultas SQL.
 - También mejoran:
 - Integridad de datos
 - Seguridad de aplicaciones
 - Confiabilidad de sistemas
 - Ventajas principales:
 - Validación automática de entradas
 - Reducción de errores de sintaxis
 - Reutilización de consultas
 - Protección frente a manipulación de datos

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Ejecutar consultas SQL parametrizadas desde Python para interactuar de forma segura con bases de datos
 - B. Evaluar ventajas de las consultas parametrizadas
 - Buenas prácticas:
 - Nunca concatenar entradas del usuario directamente en SQL
 - Utilizar siempre parámetros en consultas dinámicas
 - Implementar manejo de excepciones y validaciones
 - Resultado: aplicaciones más seguras y robustas al interactuar con bases de datos desde Python.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Comprender, gestionar y convertir tipos de datos SQL en scripts de Python
 - A. Identificar y comprender tipos de datos SQL y sus equivalentes en Python
 - Las bases de datos SQL utilizan diferentes tipos de datos para almacenar información.
 - Tipos de datos comunes en SQL:
 - INT → números enteros → Python (int)
 - FLOAT / DECIMAL → números decimales → Python (float)
 - VARCHAR / TEXT → texto → Python (str)
 - DATE / DATETIME → fechas y horas → Python (datetime)
 - BOOLEAN → valores lógicos → Python (bool)
 - Comprender estas equivalencias es útil para:
 - Integración entre Python y bases de datos
 - Procesamiento correcto de información
 - Prevención de errores de conversión
 - Aplicación práctica: análisis de datos, automatización y desarrollo de aplicaciones conectadas a bases de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Comprender, gestionar y convertir tipos de datos SQL en scripts de Python
 - B. Convertir tipos de datos adecuadamente entre SQL y Python
 - Durante la transferencia de datos pueden requerirse conversiones explícitas.
 - Ejemplos comunes:
 - Convertir texto a número
 - Transformar fechas SQL en objetos datetime
 - Convertir valores nulos (NULL) en None
 - Buenas prácticas:
 - Validar tipos antes de procesar datos
 - Manejar valores nulos correctamente
 - Evitar pérdidas de precisión en conversiones numéricas
 - Beneficios:
 - Mayor integridad de datos
 - Compatibilidad entre sistemas
 - Reducción de errores en procesos analíticos
 - Resultado: manejo eficiente y seguro de datos entre entornos SQL y aplicaciones Python.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Comprender conceptos esenciales de seguridad en bases de datos y prevención de SQL Injection
 - A. Comprender principios fundamentales de seguridad en bases de datos
 - La seguridad en bases de datos protege la información contra:
 - Accesos no autorizados
 - Manipulación de datos
 - Pérdida de información
 - Ataques maliciosos
 - Uno de los ataques más comunes es SQL Injection, donde se inserta código SQL malicioso en consultas.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Comprender conceptos esenciales de seguridad en bases de datos y prevención de SQL Injection
 - A. Comprender principios fundamentales de seguridad en bases de datos
 - Medidas básicas de seguridad:
 - Uso de consultas parametrizadas
 - Validación de entradas del usuario
 - Control de permisos y accesos
 - Uso de contraseñas seguras
 - Respaldo periódico de datos
 - Beneficios:
 - Protección de información sensible
 - Mayor confiabilidad del sistema
 - Reducción de vulnerabilidades
 - Aplicación práctica: sistemas web, aplicaciones empresariales y plataformas de análisis de datos.

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Comprender conceptos esenciales de seguridad en bases de datos y prevención de SQL Injection
 - B. Evaluar y aplicar estrategias para escribir consultas SQL seguras en Python
 - Las consultas SQL deben diseñarse para evitar manipulación externa de datos.
 - Estrategias recomendadas:
 - Utilizar consultas parametrizadas
 - Evitar concatenar texto directamente en SQL
 - Validar y **sanitizar** entradas
 - Implementar manejo de excepciones

CHAPTER 2 — PROGRAMMING AND DATABASE SKILLS

- Comprender conceptos esenciales de seguridad en bases de datos y prevención de SQL Injection
 - B. Evaluar y aplicar estrategias para escribir consultas SQL seguras en Python
 - Buenas prácticas adicionales:
 - Limitar privilegios de usuarios de base de datos
 - Registrar accesos y errores
 - Mantener librerías y sistemas actualizados
 - Resultado: aplicaciones más seguras, robustas y resistentes frente a ataques de SQL Injection y otros riesgos de seguridad.